

Министерство образования Республики Беларусь

Учреждение высшего образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Отчет по лабораторной работе № 1

Тема: «Преобразование Фурье, свертка и корреляция сигналов»

Выполнил:
студент группы 250541
Власов Р. Е.

Проверил:
Перцев Д. Ю.

Минск 2026

1 Исходные данные

Цель лабораторной работы — реализовать основные операции цифровой обработки двух периодических сигналов, выполнить преобразование Фурье ручным алгоритмом БПФ с прореживанием по частоте и проверить результаты библиотечными средствами NumPy.

Вариант лабораторной работы № 1: БПФ_Ч 0, то есть быстрое преобразование Фурье с прореживанием по частоте. В программе этот вариант реализован алгоритмом DIF FFT.

Таблица 1 – Исходные данные

Параметр	Значение
Частота дискретизации	2048 Гц
Длительность сигналов	2,0 с
Количество отсчетов	$N = M = 4096s$
Сигнал X	$0,70 \cdot \sin(2\pi \cdot 220t) + 0,40 \cdot \sin(2\pi \cdot 440t + 0,3) + 0,20 \cdot \cos(2\pi \cdot 110t + 1,0)$
Сигнал Y	$0,80 \cdot \sin(2\pi \cdot 262t + 0,1) + 0,35 \cdot \cos(2\pi \cdot 392t + 0,8) + 0,15 \cdot \sin(2\pi \cdot 524t)$
Длина полной свертки и корреляции	$N + M - 1 = 8191$ отсчет
Длина БПФ для свертки	$L_{fft} = 8192$

2 Описание алгоритма работы программы

2.1 Дискретное преобразование Фурье

Дискретное преобразование Фурье переводит сигнал из временной области в частотную. Для последовательности $x[n]$ длины N прямое и обратное преобразования задаются формулами:

$$X[k] = \sum x[n] \cdot e^{-j \cdot 2\pi kn/N}, \quad k = 0, 1, \dots, N-1$$
$$x[n] = (1/N) \cdot \sum X[k] \cdot e^{j \cdot 2\pi kn/N}, \quad n = 0, 1, \dots, N-1$$

В программе прямое преобразование реализовано функцией `fft_dif`, а обратное — через комплексное сопряжение: $\text{IFFT}(X) = \text{conj}(\text{FFT}(\text{conj}(X))) / N$. Для сравнения дополнительно используются `numpy.fft.fft` и `numpy.fft.ifft`.

Корректность обратного преобразования оценивалась отдельно: для сигналов X , Y и результата свертки после IFFT вычислялась максимальная абсолютная погрешность относительно библиотечной реализации NumPy. Полученные значения приведены в таблице 2 и подтверждают, что восстановленные временные последовательности совпадают с исходными в пределах машинной точности.

2.2 Быстрое преобразование Фурье с прореживанием по частоте

БПФ с прореживанием по частоте сокращает число операций за счет рекурсивного деления задачи на половины. На каждом шаге применяется бабочка DIF:

$$a[k] = x[k] + x[k + N/2]$$

$$a[k + N/2] = (x[k] - x[k + N/2]) \cdot W_N^k, \quad W_N^k = e^{-j \cdot 2\pi k/N}$$

После рекурсивного вычисления половин выполняется перестановка с битовой инверсией индексов, чтобы получить спектральные коэффициенты в естественном порядке. Если длина сигнала не является степенью двойки, последовательность предварительно дополняется нулями.

2.3 Линейная свертка и корреляция

Линейная свертка двух дискретных сигналов $x[n]$ и $y[n]$ вычисляется прямым суммированием. Полная корреляция реализована как свертка первого сигнала с развернутым вторым сигналом.

$$z[n] = \sum x[m] \cdot y[n - m]$$

$$r_{xy}[l] = \sum x[n] \cdot y[n - l]$$

Для проверки теоремы свертки оба сигнала дополняются нулями до длины не меньше $N + M - 1$. При отсутствии такого дополнения БПФ дает циклическую свертку, поэтому в работе выбрана длина 8192 — ближайшая степень двойки к 8191.

3 Порядок выполнения лабораторной работы

На первом этапе были заданы основные параметры эксперимента: частота дискретизации 2048 Гц и длительность каждого звукового файла 2 секунды. По этим параметрам сформирована временная сетка, содержащая 4096 отсчетов для каждого сигнала.

На втором этапе сгенерированы два периодических сигнала. Каждый сигнал является суммой нескольких гармонических составляющих с разными

частотами, амплитудами и фазовыми сдвигами. После генерации сигналы нормированы по максимальной амплитуде, чтобы избежать переполнения при записи в 16-битный WAV-файл.

На третьем этапе исходные последовательности сохранены в файлы периодический_сигнал_1.wav и периодический_сигнал_2.wav. Это позволяет не только анализировать сигналы численно, но и использовать результаты как звуковые файлы.

На четвертом этапе выполнена полная линейная свертка сигналов. В ручной реализации использовано прямое двойное суммирование: каждый отсчет первого сигнала умножается на каждый отсчет второго, а произведение добавляется в соответствующую позицию выходной последовательности длиной $N + M - 1$.

На пятом этапе выполнена полная корреляция. Для ручного расчета второй сигнал разворачивается по времени, после чего применяется та же процедура полной свертки. Такой способ позволяет получить корреляционную функцию для всех отрицательных и положительных сдвигов.

На шестом этапе реализованы библиотечные варианты тех же операций: `numpy.convolve` для свертки и `numpy.correlate` для корреляции. Полученные массивы сравниваются с ручными результатами по максимальной абсолютной погрешности.

На седьмом этапе выполнено прямое БПФ DIF для сигналов X , Y и результата свертки. Перед преобразованием последовательности дополняются нулями до ближайшей степени двойки, так как рекурсивный алгоритм БПФ работает с длинами 2^k .

На восьмом этапе выполнено обратное преобразование Фурье. Для проверки восстановления вычисляется IFFT от полученных спектров, затем результат сравнивается с библиотечной реализацией NumPy. Малые значения погрешности показывают, что прямое и обратное преобразования реализованы корректно.

На девятом этапе экспериментально проверена теорема свертки. Сигналы дополнены нулями до длины 8192, после чего свертка вычислена как $IFFT(FFT(x) \cdot FFT(y))$. Корреляция через Фурье рассчитывается аналогично, но с предварительным разворотом второго сигнала. Результаты сравниваются с прямыми алгоритмами.

На десятом этапе построены графики исходных сигналов во временной области, спектров сигналов, результата свертки, корреляции по оси сдвига и график сравнения эффективности алгоритмов. Для оценки зависимости от числа отсчетов измерено время выполнения при $N = 256, 512, 1024$ и 2048 .

На завершающем этапе сформирована сводка результатов: длины сигналов, выбранная длина БПФ, погрешности ручной и библиотечной реализаций, а также времена выполнения прямых алгоритмов и алгоритмов через БПФ.

4 Результаты работы программы

Программа формирует WAV-файлы исходных и обработанных сигналов, графики во временной и частотной областях, а также текстовую сводку с погрешностями ручной реализации относительно NumPy.

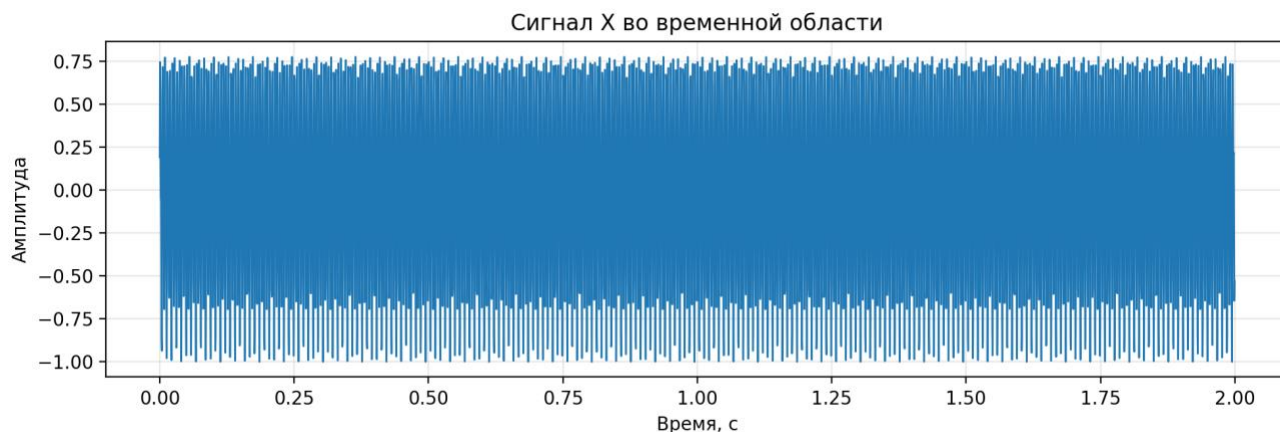


Рисунок 1 – Сигнал X во временной области

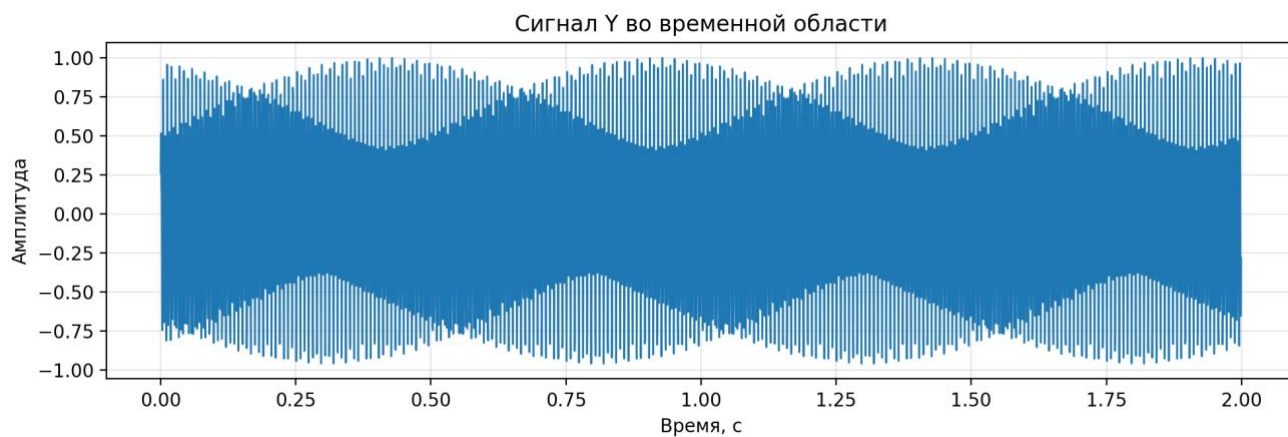


Рисунок 2 – Сигнал Y во временной области

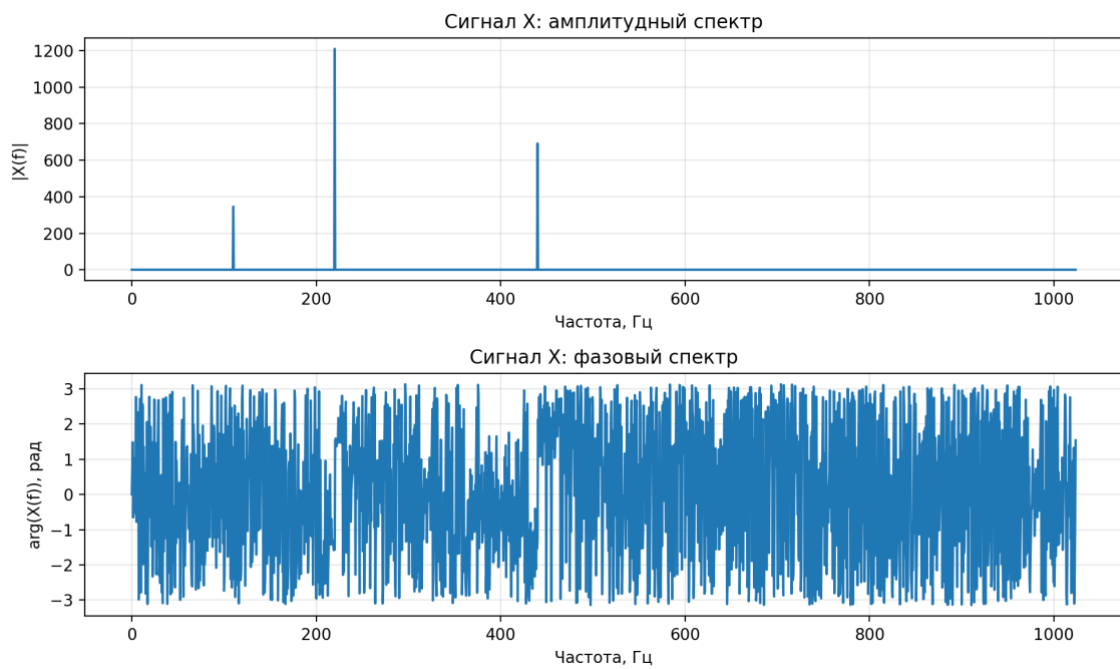


Рисунок 3 – Амплитудный и фазовый спектры сигнала X

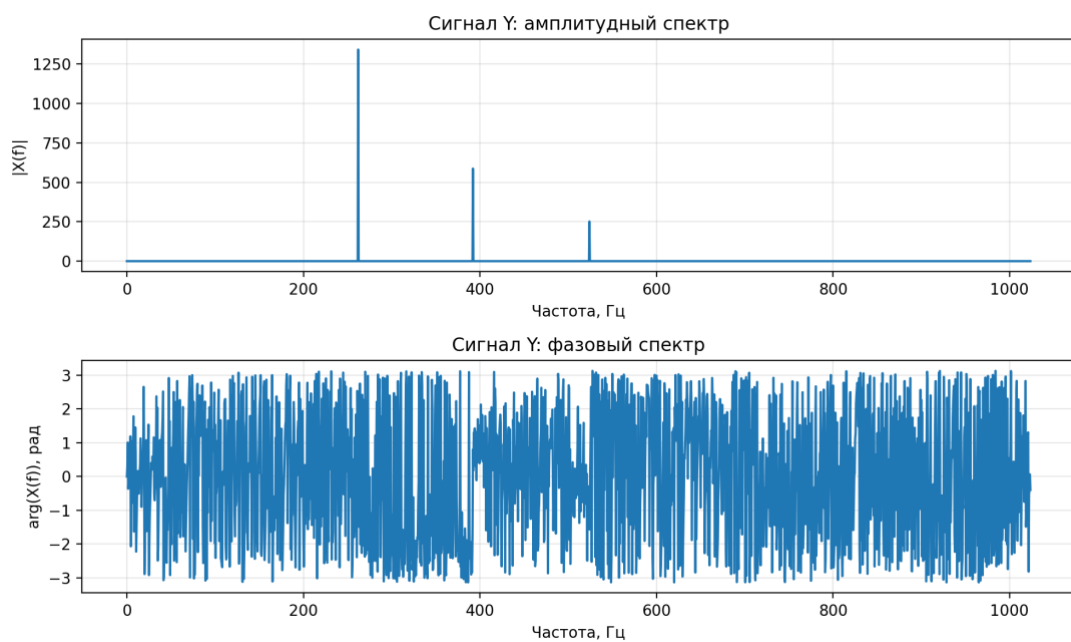


Рисунок 4 – Амплитудный и фазовый спектры сигнала Y

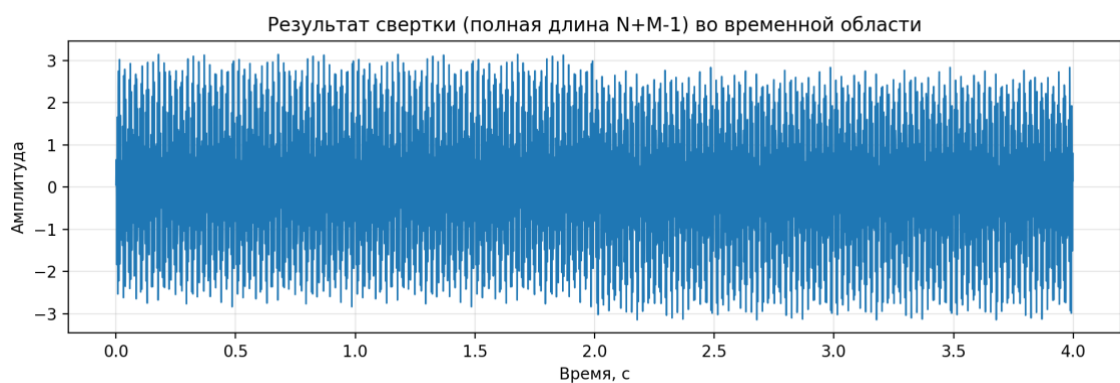


Рисунок 5 – Результат полной свертки во временной области

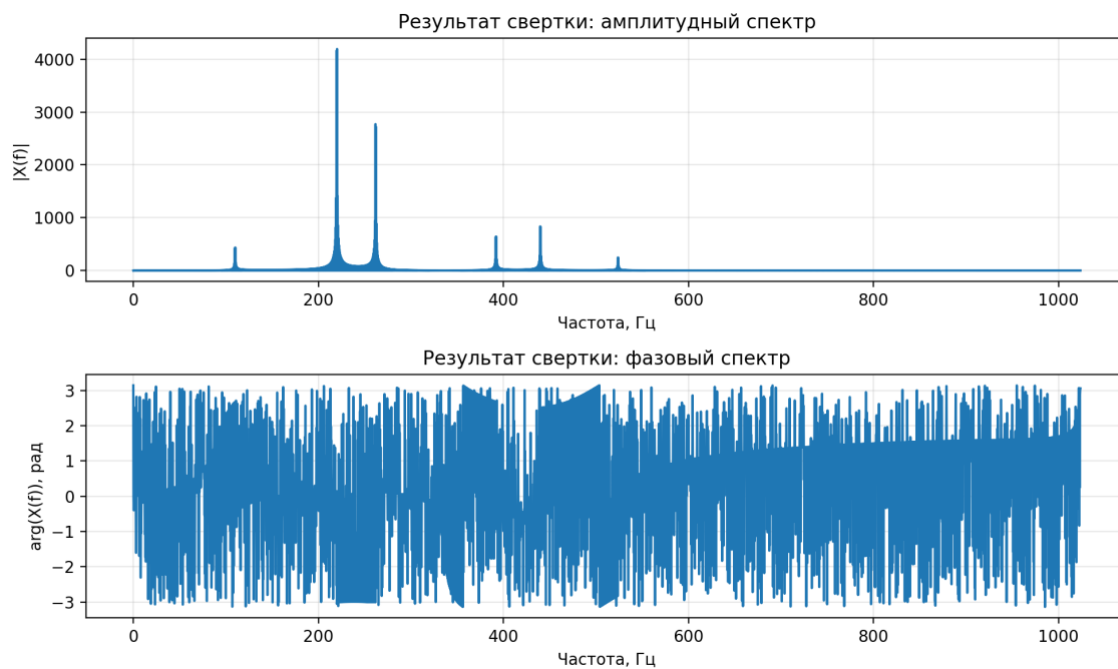


Рисунок 6 – Спектр результата свертки

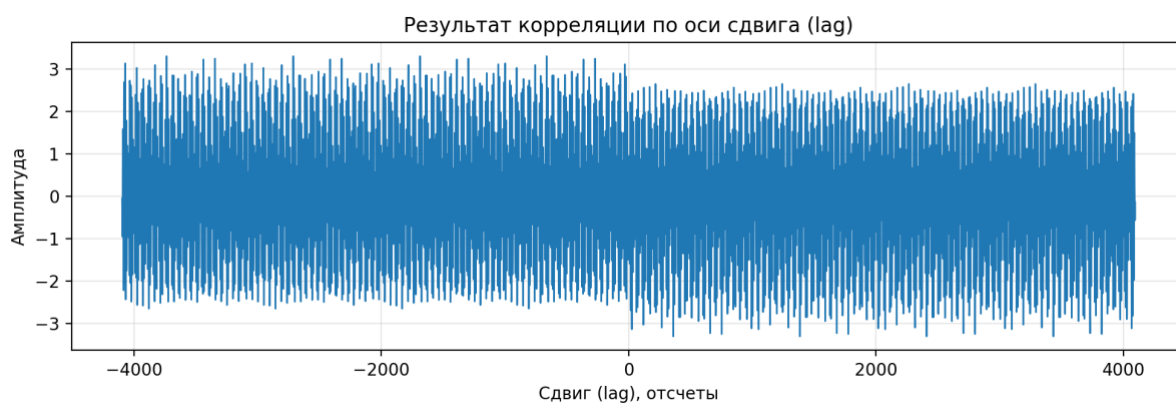


Рисунок 7 – Результат корреляции по оси сдвига

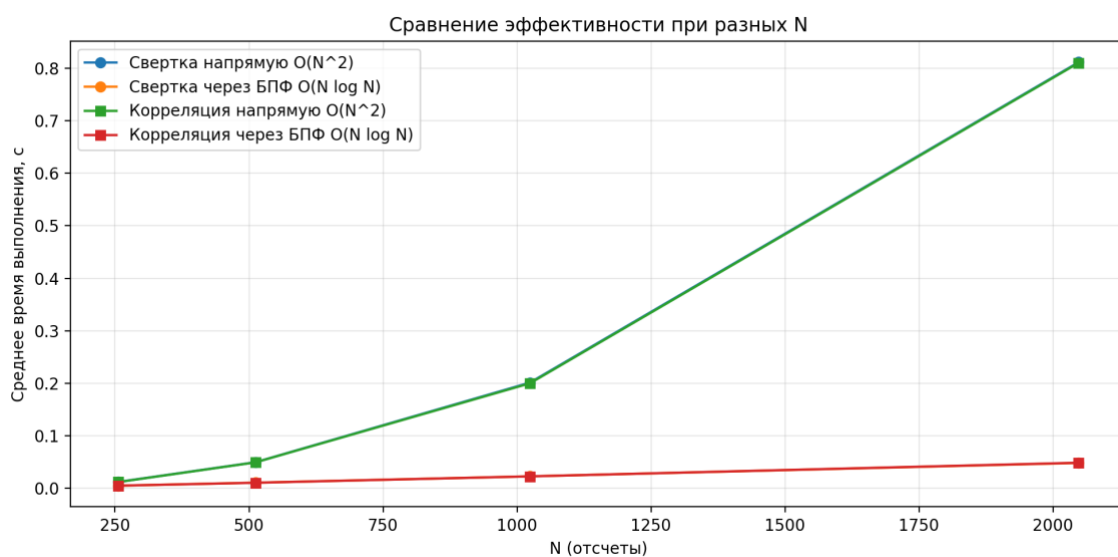


Рисунок 8 – Сравнение эффективности прямых алгоритмов и алгоритмов через БПФ

Таблица 2 – Погрешности ручной реализации относительно NumPy

Проверка	Максимальная абсолютная погрешность
FFT(X)	$5,911835 \cdot 10^{-12}$
IFFT(X)	$5,218048 \cdot 10^{-14}$
FFT(Y)	$1,968919 \cdot 10^{-11}$
IFFT(Y)	$5,551115 \cdot 10^{-14}$
FFT(Conv)	$7,869396 \cdot 10^{-11}$
IFFT(Conv)	$1,696421 \cdot 10^{-13}$
Свертка напрямую и через БПФ	$2,540190 \cdot 10^{-13}$
Корреляция напрямую и через БПФ	$2,948752 \cdot 10^{-13}$
Свертка ручная и NumPy	$1,787459 \cdot 10^{-14}$
Корреляция ручная и NumPy	$1,887379 \cdot 10^{-14}$

Таблица 3 – Результаты бенчмарка

N	Свертка напрямую, с	Свертка БПФ, с	Корреляция напрямую, с	Корреляция БПФ, с
256	0,012186	0,005006	0,011893	0,005050
512	0,049636	0,010735	0,049392	0,010601
1024	0,201251	0,022779	0,199901	0,022752
2048	0,811546	0,048319	0,810355	0,048678

5 Вывод

В ходе лабораторной работы были реализованы генерация периодических сигналов, линейная свертка, полная корреляция, прямое и обратное БПФ с прореживанием по частоте. Ручные алгоритмы были сравнены с библиотечными реализациями NumPy; максимальные погрешности имеют порядок 10^{-11} – 10^{-14} , что подтверждает корректность вычислений. Бенчмарк показал, что методы через БПФ работают быстрее прямого суммирования, особенно при увеличении числа отсчетов.

Листинг программы

Файл signal_io.py

```
001 from __future__ import annotations
002
003 from pathlib import Path
004 import wave
005
006 import numpy as np
007
008
009 INT16_SCALE = 32767
010
011
012 def normalize(signal: np.ndarray) -> np.ndarray:
013     """Нормирует сигнал так, чтобы максимальная амплитуда не превышала
014     1."""
015     peak = np.max(np.abs(signal))
016     if peak == 0:
017         return signal.astype(np.float64)
018     return (signal / peak).astype(np.float64)
019
020 def save_wav_mono_16(path: Path, sample_rate: int, signal: np.ndarray) ->
None:
021     """Сохраняет одномерный сигнал в монофонический 16-битный WAV-файл."""
022     path.parent.mkdir(parents=True, exist_ok=True)
023     normalized = normalize(signal)
024     int_data = np.clip(normalized * INT16_SCALE, -INT16_SCALE,
INT16_SCALE).astype(np.int16)
025
026     with wave.open(str(path), "wb") as wav_file:
027         wav_file.setnchannels(1)
028         wav_file.setsampwidth(2)
029         wav_file.setframerate(sample_rate)
030         wav_file.writeframes(int_data.tobytes())
031
032
033 def read_wav_mono_16(path: Path) -> tuple[int, np.ndarray]:
034     """Считывает 16-битный WAV-файл и возвращает частоту дискретизации и
массив отсчетов."""
035     with wave.open(str(path), "rb") as wav_file:
036         channels = wav_file.getnchannels()
037         sample_rate = wav_file.getframerate()
038         sample_width = wav_file.getsampwidth()
039         frame_count = wav_file.getnframes()
040         raw = wav_file.readframes(frame_count)
041
042         if sample_width != 2:
043             raise ValueError(f"Поддерживается только 16-битный PCM, получен
{sample_width * 8}-битный формат.")
044
045         data = np.frombuffer(raw, dtype=np.int16).astype(np.float64) /
INT16_SCALE
046
047         if channels > 1:
048             data = data[:, :channels]
049
050     return sample_rate, data
051
052
```

```

053 def generate_periodic_signals(sample_rate: int, duration_sec: float) ->
tuple[np.ndarray, np.ndarray]:
054     """Генерирует два периодических тестовых сигнала для лабораторной
работы."""
055     t = np.arange(0.0, duration_sec, 1.0 / sample_rate)
056
057     # Каждый сигнал составлен из нескольких гармоник с разными фазами и
амплитудами.
058     signal_x = (
059         0.70 * np.sin(2 * np.pi * 220 * t)
060         + 0.40 * np.sin(2 * np.pi * 440 * t + 0.3)
061         + 0.20 * np.cos(2 * np.pi * 110 * t + 1.0)
062     )
063     signal_y = (
064         0.80 * np.sin(2 * np.pi * 262 * t + 0.1)
065         + 0.35 * np.cos(2 * np.pi * 392 * t + 0.8)
066         + 0.15 * np.sin(2 * np.pi * 524 * t)
067     )
068
069     return normalize(signal_x), normalize(signal_y)

```

Файл `fft_dif.py`

```

001 from __future__ import annotations
002
003 import math
004 from typing import Iterable
005
006 import numpy as np
007
008
009 def next_power_of_two(n: int) -> int:
010     """Возвращает ближайшую степень двойки, не меньшую n."""
011     if n <= 1:
012         return 1
013     return 1 << (n - 1).bit_length()
014
015
016 def pad_to_power_of_two(values: Iterable[complex | float]) -> np.ndarray:
017     """Дополняет последовательность нулями до длины 2^k."""
018     arr = np.asarray(list(values), dtype=np.complex128)
019     target = next_power_of_two(len(arr))
020     if len(arr) == target:
021         return arr.copy()
022     padded = np.zeros(target, dtype=np.complex128)
023     padded[: len(arr)] = arr
024     return padded
025
026
027 def bit_reverse_index(index: int, bits: int) -> int:
028     """Вычисляет индекс с обратным порядком битов."""
029     result = 0
030     for _ in range(bits):
031         result = (result << 1) | (index & 1)
032         index >>= 1
033     return result
034
035
036 def bit_reverse_permutation(values: np.ndarray) -> np.ndarray:
037     """Переставляет элементы массива из бит-реверсивного порядка в
естественный."""
038     n = len(values)
039     bits = int(math.log2(n))

```

```

040     out = np.zeros(n, dtype=np.complex128)
041     for i in range(n):
042         out[bit_reverse_index(i, bits)] = values[i]
043     return out
044
045
046 def _fft_dif_recursive(values: np.ndarray, direction: int) -> np.ndarray:
047     """Рекурсивная часть БПФ DIF: сначала бабочки, затем деление на
половины."""
048     n = len(values)
049     if n == 1:
050         return values
051
052     half = n // 2
053     angle = -2.0 * math.pi * direction / n
054     omega_n = complex(math.cos(angle), math.sin(angle))
055     omega = 1.0 + 0.0j
056
057     # Бабочка DIF объединяет симметричные отсчеты верхней и нижней
половин.
058     for k in range(half):
059         top = values[k]
060         bottom = values[k + half]
061         values[k] = top + bottom
062         values[k + half] = (top - bottom) * omega
063         omega *= omega_n
064
065     first_half = _fft_dif_recursive(values[:half], direction)
066     second_half = _fft_dif_recursive(values[half:], direction)
067     return np.concatenate((first_half, second_half))
068
069
070 def fft_dif(signal: Iterable[complex | float]) -> np.ndarray:
071     """Выполняет прямое БПФ с прореживанием по частоте."""
072     prepared = pad_to_power_of_two(signal)
073     raw = _fft_dif_recursive(prepared.copy(), direction=1)
074     return bit_reverse_permutation(raw)
075
076
077 def ifft_dif(spectrum: Iterable[complex | float]) -> np.ndarray:
078     """Выполняет обратное БПФ через сопряжение прямого преобразования."""
079     spectrum_np = np.asarray(list(spectrum), dtype=np.complex128)
080     n = len(spectrum_np)
081     conj = np.conj(spectrum_np)
082     raw = fft_dif(conj)
083     return np.conj(raw) / n

```

Файл ops_manual.py

```

001 from __future__ import annotations
002
003 from typing import Iterable
004
005 import numpy as np
006
007
008 def convolution_full(x: Iterable[float], y: Iterable[float]) ->
np.ndarray:
009     """Вычисляет полную линейную свертку прямым двойным суммированием."""
010     x_np = np.asarray(list(x), dtype=np.float64)
011     y_np = np.asarray(list(y), dtype=np.float64)
012
013     n = len(x_np)

```

```

014     m = len(y_np)
015     out = np.zeros(n + m - 1, dtype=np.float64)
016
017     # Каждый отсчет x умножается на каждый отсчет y и добавляется в
позицию i + j.
018     for i in range(n):
019         xi = x_np[i]
020         for j in range(m):
021             out[i + j] += xi * y_np[j]
022
023     return out
024
025
026 def correlation_full(x: Iterable[float], y: Iterable[float]) ->
np.ndarray:
027     """Вычисляет полную корреляцию как свертку с развернутым вторым
сигналом."""
028     y_reversed = list(y)[::-1]
029     return convolution_full(x, y_reversed)

```

Файл ops_numpy.py

```

001 from __future__ import annotations
002
003 import numpy as np
004
005
006 def convolution_full_numpy(x: np.ndarray, y: np.ndarray) -> np.ndarray:
007     """Библиотечная реализация полной свертки для проверки ручного
алгоритма."""
008     return np.convolve(x, y, mode="full")
009
010
011 def correlation_full_numpy(x: np.ndarray, y: np.ndarray) -> np.ndarray:
012     """Библиотечная реализация полной корреляции для проверки ручного
алгоритма."""
013     return np.correlate(x, y, mode="full")
014
015
016 def fft_numpy(x: np.ndarray) -> np.ndarray:
017     """Библиотечное прямое преобразование Фурье."""
018     return np.fft.fft(x)
019
020
021 def ifft_numpy(x: np.ndarray) -> np.ndarray:
022     """Библиотечное обратное преобразование Фурье."""
023     return np.fft.ifft(x)

```

Файл ops_fft_check.py

```

001 from __future__ import annotations
002
003 from typing import Iterable
004
005 import numpy as np
006
007 from lab1.fft_dif import fft_dif, ifft_dif, next_power_of_two
008
009
010 def _pad_to_length(signal: np.ndarray, length: int) -> np.ndarray:
011     """Дополняет сигнал нулями до заданной длины БПФ."""
012     out = np.zeros(length, dtype=np.float64)

```

```

013     out[: len(signal)] = signal
014     return out
015
016
017 def convolution_via_fft_dif(x: Iterable[float], y: Iterable[float]) ->
np.ndarray:
018     """Вычисляет линейную свертку через теорему Фурье и ручной БПФ DIF."""
019     x_np = np.asarray(list(x), dtype=np.float64)
020     y_np = np.asarray(list(y), dtype=np.float64)
021
022     # Для линейной свертки длина БПФ должна быть не меньше N + M - 1.
023     linear_size = len(x_np) + len(y_np) - 1
024     fft_size = next_power_of_two(linear_size)
025
026     x_pad = _pad_to_length(x_np, fft_size)
027     y_pad = _pad_to_length(y_np, fft_size)
028
029     X = fft_dif(x_pad)
030     Y = fft_dif(y_pad)
031
032     # По теореме свертки произведение спектров соответствует свертке во
времени.
033     result = ifft_dif(X * Y)
034     return np.real(result[:linear_size])
035
036
037 def correlation_via_fft_dif(x: Iterable[float], y: Iterable[float]) ->
np.ndarray:
038     """Вычисляет корреляцию через свертку с развернутым вторым
сигналом."""
039     y_reversed = np.asarray(list(y), dtype=np.float64)[::-1]
040     return convolution_via_fft_dif(x, y_reversed)

```

Файл benchmark.py

```

001 from __future__ import annotations
002
003 import time
004 from typing import Callable, Iterable
005
006 import numpy as np
007
008
009 MeasureFn = Callable[[Iterable[float], Iterable[float]], np.ndarray]
010
011
012 def _measure_seconds(fn: MeasureFn, x: np.ndarray, y: np.ndarray, repeats:
int = 3) -> float:
013     """Измеряет среднее время выполнения функции на нескольких
повторах."""
014     timings: list[float] = []
015     for _ in range(repeats):
016         start = time.perf_counter()
017         fn(x, y)
018         timings.append(time.perf_counter() - start)
019     return float(np.mean(timings))
020
021
022 def benchmark_pair(
023     x: np.ndarray,
024     y: np.ndarray,
025     direct_fn: MeasureFn,
026     fft_fn: MeasureFn,

```

```

027     sizes: list[int],
028 ) -> tuple[list[int], list[float], list[float]]:
029     """Сравнивает прямой алгоритм и алгоритм через БПФ для набора длин
N."""
030     direct_times: list[float] = []
031     fft_times: list[float] = []
032
033     # Для каждого N берутся первые N отсчетов исходных сигналов.
034     for n in sizes:
035         x_n = x[:n]
036         y_n = y[:n]
037         direct_times.append(_measure_seconds(direct_fn, x_n, y_n))
038         fft_times.append(_measure_seconds(fft_fn, x_n, y_n))
039
040     return sizes, direct_times, fft_times

```

Файл plotting.py

```

001 from __future__ import annotations
002
003 from pathlib import Path
004 from typing import Callable
005
006 import matplotlib
007
008 matplotlib.use("Agg")
009
010 import matplotlib.pyplot as plt
011 import numpy as np
012
013
014 FFTFunction = Callable[[np.ndarray], np.ndarray]
015
016
017 def _ensure_dir(path: Path) -> None:
018     """Создает папку для графика, если она еще не существует."""
019     path.parent.mkdir(parents=True, exist_ok=True)
020
021
022 def plot_time_domain(
023     signal: np.ndarray,
024     sample_rate: int,
025     path: Path,
026     title: str,
027     max_samples: int | None = None,
028 ) -> None:
029     """Строит график сигнала во временной области."""
030     _ensure_dir(path)
031     samples = len(signal) if max_samples is None else min(max_samples,
len(signal))
032     time_axis = np.arange(samples) / sample_rate
033
034     plt.figure(figsize=(10, 3.5))
035     plt.plot(time_axis, signal[:samples], linewidth=1.0)
036     plt.title(title)
037     plt.xlabel("Время, с")
038     plt.ylabel("Амплитуда")
039     plt.grid(True, alpha=0.3)
040     plt.tight_layout()
041     plt.savefig(path, dpi=200)
042     plt.close()
043
044

```

```

045 def plot_amplitude_phase_spectrum(
046     signal: np.ndarray,
047     sample_rate: int,
048     fft_fn: FFTFunction,
049     path: Path,
050     title: str,
051 ) -> None:
052     """Строит амплитудный и фазовый спектры сигнала."""
053     _ensure_dir(path)
054     spectrum = fft_fn(signal)
055     n = len(spectrum)
056
057     frequencies = np.arange(n) * sample_rate / n
058     amplitude = np.abs(spectrum)
059     phase = np.angle(spectrum)
060
061     half = n // 2
062     frequencies = frequencies[:half]
063     amplitude = amplitude[:half]
064     phase = phase[:half]
065
066     plt.figure(figsize=(10, 6))
067
068     plt.subplot(2, 1, 1)
069     plt.plot(frequencies, amplitude)
070     plt.title(f"{title}: амплитудный спектр")
071     plt.xlabel("Частота, Гц")
072     plt.ylabel("|X(f)|")
073     plt.grid(True, alpha=0.3)
074
075     plt.subplot(2, 1, 2)
076     plt.plot(frequencies, phase)
077     plt.title(f"{title}: фазовый спектр")
078     plt.xlabel("Частота, Гц")
079     plt.ylabel("arg(X(f)), рад")
080     plt.grid(True, alpha=0.3)
081
082     plt.tight_layout()
083     plt.savefig(path, dpi=200)
084     plt.close()
085
086
087 def plot_correlation_by_lag(
088     correlation: np.ndarray,
089     len_x: int,
090     len_y: int,
091     path: Path,
092     title: str,
093     max_points: int | None = None,
094 ) -> None:
095     """Строит график полной корреляции по оси сдвига."""
096     _ensure_dir(path)
097     lags = np.arange(-(len_y - 1), len_x)
098
099     corr = correlation
100     lag_axis = lags
101     if max_points is not None and len(corr) > max_points:
102         step = int(np.ceil(len(corr) / max_points))
103         corr = corr[::step]
104         lag_axis = lags[::step]
105
106     plt.figure(figsize=(10, 3.5))
107     plt.plot(lag_axis, corr, linewidth=1.0)
108     plt.title(title)

```

```

109     plt.xlabel("Сдвиг (lag), отсчеты")
110     plt.ylabel("Амплитуда")
111     plt.grid(True, alpha=0.3)
112     plt.tight_layout()
113     plt.savefig(path, dpi=200)
114     plt.close()
115
116
117 def plot_benchmark(
118     sizes: list[int],
119     direct_conv: list[float],
120     fft_conv: list[float],
121     direct_corr: list[float],
122     fft_corr: list[float],
123     path: Path,
124 ) -> None:
125     """Строит график сравнения времени прямых алгоритмов и алгоритмов
126     через БПФ."""
127     _ensure_dir(path)
128     plt.figure(figsize=(10, 5))
129
130     plt.plot(sizes, direct_conv, "o-", label="Свертка напрямую  $O(N^2)$ ")
131     plt.plot(sizes, fft_conv, "o-", label="Свертка через БПФ  $O(N \log N)$ ")
132     plt.plot(sizes, direct_corr, "s-", label="Корреляция напрямую  $O(N^2)$ ")
133     plt.plot(sizes, fft_corr, "s-", label="Корреляция через БПФ  $O(N \log N)$ ")
134
135     plt.xlabel("N (отсчеты)")
136     plt.ylabel("Среднее время выполнения, с")
137     plt.title("Сравнение эффективности при разных N")
138     plt.grid(True, alpha=0.3)
139     plt.legend()
140     plt.tight_layout()
141     plt.savefig(path, dpi=200)
142     plt.close()

```

Файл main.py

```

001 from __future__ import annotations
002
003 from pathlib import Path
004
005 import numpy as np
006
007 from lab1.benchmark import benchmark_pair
008 from lab1.fft_dif import fft_dif, ifft_dif, pad_to_power_of_two
009 from lab1.ops_fft_check import convolution_via_fft_dif,
010 correlation_via_fft_dif
011 from lab1.ops_manual import convolution_full, correlation_full
012 from lab1.ops_numpy import (
013     convolution_full_numpy,
014     correlation_full_numpy,
015     fft_numpy,
016     ifft_numpy,
017 )
018 from lab1.plotting import (
019     plot_amplitude_phase_spectrum,
020     plot_benchmark,
021     plot_correlation_by_lag,
022     plot_time_domain,
023 )
024 from lab1.signal_io import generate_periodic_signals, save_wav_mono_16
025
026 ROOT = Path(__file__).resolve().parent

```

```

027 OUTPUT = ROOT / "output"
028
029 SAMPLE_RATE = 2_048
030 DURATION_SEC = 2.0
031
032
033 def _max_abs_error(a: np.ndarray, b: np.ndarray) -> float:
034     """Возвращает максимальную абсолютную погрешность между двумя
массивами."""
035     n = min(len(a), len(b))
036     return float(np.max(np.abs(a[:n] - b[:n])))
037
038
039 def _manual_fft(signal: np.ndarray) -> np.ndarray:
040     """Обертка для построения спектров ручной реализацией БПФ DIF."""
041     return fft_dif(pad_to_power_of_two(signal))
042
043
044 def run() -> None:
045     OUTPUT.mkdir(parents=True, exist_ok=True)
046
047     # 1. Генерация двух периодических сигналов и сохранение их в WAV-
файлы.
048     signal_x, signal_y = generate_periodic_signals(SAMPLE_RATE,
DURATION_SEC)
049
050     save_wav_mono_16(OUTPUT / "периодический_сигнал_1.wav", SAMPLE_RATE,
signal_x)
051     save_wav_mono_16(OUTPUT / "периодический_сигнал_2.wav", SAMPLE_RATE,
signal_y)
052
053     # 2. Ручные и библиотечные операции свертки и корреляции.
054     conv_manual = convolution_full(signal_x, signal_y)
055     corr_manual = correlation_full(signal_x, signal_y)
056     conv_numpy = convolution_full_numpy(signal_x, signal_y)
057     corr_numpy = correlation_full_numpy(signal_x, signal_y)
058
059     save_wav_mono_16(OUTPUT / "сигнал_после_свертки.wav", SAMPLE_RATE,
conv_manual)
060     save_wav_mono_16(OUTPUT / "сигнал_после_корреляции.wav", SAMPLE_RATE,
corr_manual)
061
062     # 3. Проверка теоремы Фурье для свертки и корреляции.
063     conv_from_fft = convolution_via_fft_dif(signal_x, signal_y)
064     corr_from_fft = correlation_via_fft_dif(signal_x, signal_y)
065
066     # 4. Подготовка сигналов к БПФ: длины должны быть степенями двойки.
067     x_pad = pad_to_power_of_two(signal_x)
068     y_pad = pad_to_power_of_two(signal_y)
069     conv_pad = pad_to_power_of_two(conv_manual)
070     linear_conv_length = len(signal_x) + len(signal_y) - 1
071     fft_conv_length = 1 << (linear_conv_length - 1).bit_length()
072
073     x_manual_fft = fft_dif(x_pad)
074     y_manual_fft = fft_dif(y_pad)
075     conv_manual_fft = fft_dif(conv_pad)
076
077     # 5. Обратное БПФ проверяет восстановление сигналов из спектра.
078     x_manual_ifft = ifft_dif(x_manual_fft)
079     y_manual_ifft = ifft_dif(y_manual_fft)
080     conv_manual_ifft = ifft_dif(conv_manual_fft)
081
082     # 6. Библиотечные FFT/IFFT нужны для численного сравнения.
083     x_numpy_fft = fft_numpy(x_pad)

```

```

084     y_numpy_fft = fft_numpy(y_pad)
085     conv_numpy_fft = fft_numpy(conv_pad)
087     x_numpy_ifft = ifft_numpy(x_numpy_fft)
088     y_numpy_ifft = ifft_numpy(y_numpy_fft)
089     conv_numpy_ifft = ifft_numpy(conv_numpy_fft)
090
091     # 7. Построение графиков во временной и частотной областях.
092     plot_time_domain(
093         signal_x,
094         SAMPLE_RATE,
095         OUTPUT / "сигнал_x_временная_область.png",
096         "Сигнал X во временной области",
097         max_samples=None,
098     )
099     plot_time_domain(
100         signal_y,
101         SAMPLE_RATE,
102         OUTPUT / "сигнал_y_временная_область.png",
103         "Сигнал Y во временной области",
104         max_samples=None,
105     )
106     plot_time_domain(
107         conv_manual,
108         SAMPLE_RATE,
109         OUTPUT / "результат_свертки_временная_область.png",
110         "Результат свертки (полная длина N+M-1) во временной области",
111         max_samples=None,
112     )
113     plot_correlation_by_lag(
114         corr_manual,
115         len(signal_x),
116         len(signal_y),
117         OUTPUT / "результат_корреляции_сдвиг.png",
118         "Результат корреляции по оси сдвига (lag)",
119     )
120
121     plot_amplitude_phase_spectrum(signal_x, SAMPLE_RATE, _manual_fft,
122     OUTPUT / "сигнал_x_спектр.png", "Сигнал X")
123     plot_amplitude_phase_spectrum(signal_y, SAMPLE_RATE, _manual_fft,
124     OUTPUT / "сигнал_y_спектр.png", "Сигнал Y")
125     plot_amplitude_phase_spectrum(conv_manual, SAMPLE_RATE, _manual_fft,
126     OUTPUT / "результат_свертки_спектр.png", "Результат свертки")
127     # 8. Бенчмарк показывает зависимость времени работы от количества
128     # отсчетов N.
129     sizes = [256, 512, 1024, 2048]
130     sizes, direct_conv, fft_conv = benchmark_pair(
131         signal_x,
132         signal_y,
133         convolution_full,
134         convolution_via_fft_dif,
135         sizes,
136     )
137     _, direct_corr, fft_corr = benchmark_pair(
138         signal_x,
139         signal_y,
140         correlation_full,
141         correlation_via_fft_dif,
142         sizes,
143     )
144     plot_benchmark(
145         sizes,
146         direct_conv,
147         fft_conv,
148         direct_corr,

```

```

146         fft_corr,
147         OUTPUT / "сравнение_эффективности_алгоритмов.png",
148     )
149     # 9. Текстовая сводка фиксирует параметры эксперимента, ошибки и время
150     # выполнения.
151     report_lines = [
152         "Сводка по лабораторной работе 1",
153         f"Частота дискретизации: {SAMPLE_RATE} Гц",
154         f"Длительность: {DURATION_SEC} с",
155         "",
156         "Длины сигналов и временные оси:",
157         f"N = {len(signal_x)}, M = {len(signal_y)}, N+M-1 = {linear_conv_length}",
158         f"Длительность X: {len(signal_x) / SAMPLE_RATE:.6f} с",
159         f"Длительность Y: {len(signal_y) / SAMPLE_RATE:.6f} с",
160         f"Длительность полной свертки: {len(conv_manual) / SAMPLE_RATE:.6f} с",
161         "",
162         "Zero-padding для свертки через БПФ:",
163         "Чтобы через БПФ получить линейную свертку (а не циклическую),",
164         "оба сигнала дополняются нулями до длины Lfft >= N+M-1.",
165         f"N+M-1 = {linear_conv_length}, выбранная длина БПФ Lfft = {fft_conv_length}",
166         "",
167         "Ошибки: ручная реализация vs NumPy",
168         f"FFT(X): {_max_abs_error(x_manual_fft, x_numpy_fft):.6e}",
169         f"IFFT(X): {_max_abs_error(np.real(x_manual_ifft), np.real(x_numpy_ifft)):.6e}",
170         f"FFT(Y): {_max_abs_error(y_manual_fft, y_numpy_fft):.6e}",
171         f"IFFT(Y): {_max_abs_error(np.real(y_manual_ifft), np.real(y_numpy_ifft)):.6e}",
172         f"FFT(Conv): {_max_abs_error(conv_manual_fft, conv_numpy_fft):.6e}",
173         f"IFFT(Conv): {_max_abs_error(np.real(conv_manual_ifft), np.real(conv_numpy_ifft)):.6e}",
174         "",
175         "Проверка свертки/корреляции через теорему Фурье",
176         f"Свертка напрямую vs через БПФ: {_max_abs_error(conv_manual, conv_from_fft):.6e}",
177         f"Корреляция напрямую vs через БПФ: {_max_abs_error(corr_manual, corr_from_fft):.6e}",
178         "",
179         "Свертка/корреляция: ручная реализация vs NumPy",
180         f"Свертка ручная vs NumPy: {_max_abs_error(conv_manual, conv_numpy):.6e}",
181         f"Корреляция ручная vs NumPy: {_max_abs_error(corr_manual, corr_numpy):.6e}",
182         "",
183         "Результаты бенчмарка (в секундах):",
184     ]
185     for index, n in enumerate(sizes):
186         report_lines.append(
187             f"N={n}: свертка_напрямую={direct_conv[index]:.6f}, свертка_БПФ={fft_conv[index]:.6f}, "
188             f"корреляция_напрямую={direct_corr[index]:.6f}, корреляция_БПФ={fft_corr[index]:.6f}"
189         )
190     (OUTPUT / "сводка_результатов.txt").write_text("\n".join(report_lines), encoding="utf-8")
191     print("Лабораторная работа 1 выполнена.")
192     print(f"Результаты сохранены в: {OUTPUT}")
193
194 if __name__ == "__main__":
195     run()

```